Project Design Phase-II | Technology Stack (Architecture & Stack)

Date: 27-10-2026 | **Team ID:** SWUID2026403391 | **Project Name:** Agri-Tech | **Maximum Marks:** 4 Marks

Technical Architecture:

The Agri-Tech platform is designed with a scalable 3-tier architecture comprising the presentation layer (React.js frontend), the business logic layer (Node.js and Express.js backend), and the data storage layer (MongoDB). The solution ensures performance, security, and ease of integration with third-party APIs such as a weather and pest data service.

Table-1: Components & Technologies:

S.No	Component	Description	Technology
1	User Interface	Responsive, multilingual web interface with farm dashboards, crop tracking, charts and alerts	HTML, CSS, JavaScript / React.js, Bootstrap / Material UI
2	Application Logic-1	User registration and login, farm profile, crop monitoring, CRUD operations and validation	Node.js, Express.js
3	Application Logic-2	Weather forecasting, pest alerts, soil analysis, order tracking, expert consultation and admin dashboard	Node.js, Express.js, Axios, React.js
4	Database	Stores users, farms, crops, orders and weather data	MongoDB (Atlas / Local)
5	Authentication & Security	Token-based authentication, role-based access for farmers and admins, password encryption	JWT, bcrypt
6	External API	Real-time weather forecast and pest alert data	Weather REST API via Axios
7	Testing & Version Control	API testing and collaborative development	Postman, Git, GitHub

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Frontend framework, backend framework and UI libraries	React.js, Node.js, Express.js, Bootstrap, Material UI
2	Security Implementations	Encrypted passwords, token-based sessions and admin-only sensitive operations	JWT, bcrypt, role-based access control
3	Scalable Architecture	3-tier architecture with RESTful APIs and modular controllers	MVC, REST
4	Availability	Cloud-hosted database with managed backups	MongoDB Atlas
5	Performance	Faster rendering and real-time updates on low-bandwidth rural networks	Lazy loading, code splitting, caching

References:

React.js Documentation
Node.js Getting Started
Express.js Documentation
MongoDB Documentation